

1. Dynamic Student Manager App

```
import React, { useState } from 'react';

function StudentManager() {

  const [students, setStudents] = useState([]);

  const [form, setForm] = useState({ name: "", dept: "" });

  const addStudent = (e) => {

    e.preventDefault();

    if (form.name === "" || form.dept === "") {

      alert("Fields cannot be empty!");

      return;

    }

    setStudents([...students, form]);

    setForm({ name: "", dept: "" }); // Reset form

  };

  return (

    <div style={{ padding: '20px' }}>

      <h2>Student Manager</h2>

      <form onSubmit={addStudent}>

        <input

          placeholder="Name"

          value={form.name}

          onChange={(e) => setForm({...form, name: e.target.value})}>
```

```

    />

    <input
      placeholder="Department"
      value={form.dept}
      onChange={(e) => setForm({...form, dept: e.target.value})}
    />

    <button type="submit">Add Student</button>
  </form>

  <table border="1" style={{ marginTop: '20px', width: '100%', textAlign: 'left' }}>
    <thead>
      <tr><th>Name</th><th>Department</th></tr>
    </thead>
    <tbody>
      {students.map((s, index) => (
        <tr key={index}><td>{s.name}</td><td>{s.dept}</td></tr>
      ))}
    </tbody>
  </table>
</div>

);
}

export default StudentManager;

```

2. Smart Feedback Analyzer App

```
import React, { useState } from 'react';

function FeedbackApp() {
  const [feedbacks, setFeedbacks] = useState([]);
  const [selectedRating, setSelectedRating] = useState("");
  const [name, setName] = useState("");

  const submitFeedback = () => {
    if (!name || !selectedRating) return alert("Fill all fields");
    setFeedbacks([...feedbacks, selectedRating]);
    setName("");
    setSelectedRating("");
  };

  const getCount = (rating) => feedbacks.filter(f => f === rating).length;

  return (
    <div style={{ padding: '20px' }}>
      <h3>Feedback System</h3>
      <input value={name} onChange={(e) => setName(e.target.value)} placeholder="Your Name" />
      <div>
        {[ 'Excellent', 'Good', 'Poor' ].map(r => (
          <label key={r} style={{ margin: '5px', color: selectedRating === r ? 'blue' : 'black' }}>
```

```

        <input type="radio" name="rating" value={r} onChange={(e) =>
setSelectedRating(e.target.value)} /> {r}

        </label>

    )))
</div>

<button onClick={submitFeedback}>Submit</button>

<h4>Dashboard</h4>

<p>Total Responses: {feedbacks.length}</p>

<p>Excellent: {getCount('Excellent')} | Good: {getCount('Good')} | Poor:
{getCount('Poor')}</p>

</div>

);
}

```

3. Multi-Step Registration App

```

import React, { useState } from 'react';

function RegistrationForm() {
  const [step, setStep] = useState(1);

  const [formData, setFormData] = useState({ name: "", email: "", course: "" });

  const nextStep = () => {
    if (step === 1 && (!formData.name || !formData.email)) return alert("Fill details");
    setStep(step + 1);
  };
}

```

```
if (step === 3) return (  
  <div>  
    <h3>Summary</h3>  
    <p>Name: {formData.name}</p>  
    <p>Email: {formData.email}</p>  
    <p>Course: {formData.course}</p>  
  </div>  
);
```

```
return (  
  <div style={{ padding: '20px' }}>  
    <div style={{ fontWeight: 'bold' }}>Step {step} of 2</div>  
    {step === 1 ? (  
      <div>  
        <input placeholder="Name" onChange={e => setFormData({...formData, name:  
e.target.value})} />  
        <input placeholder="Email" onChange={e => setFormData({...formData, email:  
e.target.value})} />  
      </div>  
    ) : (  
      <select onChange={e => setFormData({...formData, course: e.target.value})}>  
        <option value="">Select Course</option>  
        <option value="React">React</option>  
        <option value="Node">Node</option>  
      </select>
```

```

    })

    <button onClick={nextStep}>{step === 2 ? 'Submit' : 'Next'}</button>

  </div>

);
}

```

4. Advanced Counter Dashboard

```

import React, { useState } from 'react';

function CounterDashboard() {
  const [count, setCount] = useState(0);

  const getStatus = () => {
    if (count < 5) return "Low";
    if (count < 10) return "Medium";
    return "High";
  };

  return (
    <div style={{ textAlign: 'center', padding: '20px' }}>
      <h1>{count}</h1>
      <p>Status: <strong>{getStatus()}</strong></p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)} disabled={count ===
0}>Decrement</button>
    </div>
  );
}

```

```
    <button onClick={() => setCount(0)}>Reset</button>
  </div>

);
}
```

5. Mini Course Portal (Routing Based)

```
import { BrowserRouter, Routes, Route, Link, useParams } from 'react-router-dom';
```

```
const courses = [
  { id: 1, name: 'React Basics', desc: 'Learn hooks and components.' },
  { id: 2, name: 'Advanced JS', desc: 'Closures, Async/Await and more.' }
];
```

```
const Home = () => <h2>Welcome to Course Portal</h2>;
```

```
const Profile = () => <h2>User Profile Page</h2>;
```

```
const Courses = () => (
  <div>
    <h2>Courses List</h2>
    {courses.map(c => <li key={c.id}><Link to={` /course/${c.id}`}>{c.name}</Link></li>)}
  </div>
);
```

```
const CourseDetails = () => {
  const { id } = useParams();
```

```

const course = courses.find(c => c.id === parseInt(id));

return (
  <div>
    <h3>{course?.name}</h3>
    <p>{course?.desc}</p>
  </div>
);
};

export default function App() {
  return (
    <BrowserRouter>
      <nav style={{ background: '#eee', padding: '10px' }}>
        <Link to="/">Home</Link> | <Link to="/courses">Courses</Link> | <Link
to="/profile">Profile</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/courses" element={<Courses />} />
        <Route path="/course/:id" element={<CourseDetails />} />
        <Route path="/profile" element={<Profile />} />
      </Routes>
    </BrowserRouter>
  );
}

```


6.Design a login form using HTML and CSS that contains fields for username and password, a “Remember me” checkbox, and a “Login” button. Apply CSS to center the form on the page, add padding, rounded corners, and a hover effect for the button

Create a registration form in HTML with fields for full name, email, password, confirm password, and gender (using radio buttons), along with a “Register” button. Use CSS to style the form with proper spacing, borders for inputs, focus effects on form fields, and different background colors for the form header and body.

```
<!-- Login Form -->
<div class="form-box">
  <h2>Login</h2>
  <form>
    <input type="text" placeholder="Username" required>
    <input type="password" placeholder="Password" required>

    <div class="remember">
      <input type="checkbox" id="remember">
      <label for="remember">Remember me</label>
    </div>

    <button type="submit">Login</button>
  </form>
</div>

<!-- Registration Form -->
<div class="form-box">
  <div class="register-header">
    <h2>Register</h2>
  </div>
  <div class="register-body">
    <form>
      <input type="text" placeholder="Full Name" required>
      <input type="email" placeholder="Email" required>
      <input type="password" placeholder="Password" required>
      <input type="password" placeholder="Confirm Password" required>
```

```

    <div class="gender">
      <label><input type="radio" name="gender"> Male</label>
      <label><input type="radio" name="gender"> Female</label>
    </div>

    <button type="submit">Register</button>
  </form>
</div>
</div>

```

7. Create a responsive **contact form** using HTML and an **external CSS file** (linked via `<link>` tag) that includes fields for name, email, subject, message (textarea), and a submit button. Style the form with a clean design featuring a card-like container with shadow effects.

```

<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Contact Form</title>
<link rel="stylesheet" href="styles.css">
</head>
<body>

<div class="container">
<div class="card">
<h2>Contact Us</h2>
<form>
<input type="text" placeholder="Full Name" required>
<input type="email" placeholder="Email Address" required>
<input type="text" placeholder="Subject" required>
<textarea rows="5" placeholder="Your Message" required></textarea>
<button type="submit">Send Message</button>
</form>
</div>
</div>

```

```
</body>
</html>
```

```
/* styles.css */
body {
  font-family: Arial, sans-serif;
  background: #f4f6f9;
  margin: 0;
}
```

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  padding: 20px;
}
```

```
.card {
  background: #ffffff;
  padding: 25px;
  border-radius: 12px;
  box-shadow: 0 6px 15px rgba(0,0,0,0.1);
  width: 100%;
  max-width: 400px;
}
```

```
.card h2 {
  text-align: center;
  margin-bottom: 20px;
}
```

```
form input,
form textarea {
  width: 100%;
  padding: 10px;
  margin: 10px 0;
  border: 1px solid #ccc;
```

```
border-radius: 6px;  
font-size: 14px;  
transition: border 0.3s;  
}
```

```
form input:focus,  
form textarea:focus {  
border-color: #007bff;  
outline: none;  
}
```

```
button {  
width: 100%;  
padding: 12px;  
background: #007bff;  
color: white;  
border: none;  
border-radius: 6px;  
cursor: pointer;  
font-size: 16px;  
transition: background 0.3s;  
}
```

```
button:hover {  
background: #0056b3;  
}
```

```
/* Responsive Design */  
@media (max-width: 500px) {  
.card {  
padding: 20px;  
}
```

```
.card h2 {  
font-size: 20px;  
}  
}
```

8.Design a **contact form** for a website using HTML and CSS that includes fields for full name, email address, phone number, subject dropdown, message textarea, and submit button. Style the form with a modern CSS effect, labels for inputs, file upload for attachments.

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Modern Contact Form</title>

<style>

* {

    box-sizing: border-box;

    font-family: Arial, sans-serif;

}

body {

    margin: 0;

    background: linear-gradient(135deg, #667eea, #764ba2);

    display: flex;

    justify-content: center;

    align-items: center;

    height: 100vh;

}
```

```
.form-container {  
    background: #fff;  
    padding: 30px;  
    border-radius: 15px;  
    width: 100%;  
    max-width: 500px;  
    box-shadow: 0 10px 25px rgba(0,0,0,0.2);  
    animation: fadeIn 0.8s ease-in-out;  
}
```

```
@keyframes fadeIn {  
    from { opacity: 0; transform: translateY(20px); }  
    to { opacity: 1; transform: translateY(0); }  
}
```

```
h2 {  
    text-align: center;  
    margin-bottom: 20px;  
}
```

```
.form-group {  
    margin-bottom: 15px;  
}
```

```
label {  
    display: block;
```

```
margin-bottom: 5px;
font-weight: bold;
}
```

```
input, select, textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 8px;
  transition: all 0.3s ease;
}
```

```
input:focus, select:focus, textarea:focus {
  border-color: #667eea;
  box-shadow: 0 0 5px rgba(102,126,234,0.5);
  outline: none;
}
```

```
textarea {
  resize: none;
}
```

```
input[type="file"] {
  padding: 5px;
  border: none;
}
```

```
button {  
    width: 100%;  
    padding: 12px;  
    border: none;  
    border-radius: 8px;  
    background: #667eea;  
    color: #fff;  
    font-size: 16px;  
    cursor: pointer;  
    transition: transform 0.2s ease, background 0.3s;  
}
```

```
button:hover {  
    background: #5a67d8;  
    transform: translateY(-2px);  
}
```

```
button:active {  
    transform: scale(0.98);  
}
```

```
@media (max-width: 600px) {  
    .form-container {  
        padding: 20px;  
    }  
}
```



```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="form-container">
```

```
<h2>Contact Us</h2>
```

```
<form>
```

```
<div class="form-group">
```

```
<label for="name">Full Name</label>
```

```
<input type="text" id="name" placeholder="Enter your full name" required>
```

```
</div>
```

```
<div class="form-group">
```

```
<label for="email">Email Address</label>
```

```
<input type="email" id="email" placeholder="Enter your email" required>
```

```
</div>
```

```
<div class="form-group">
```

```
<label for="phone">Phone Number</label>
```

```
<input type="tel" id="phone" placeholder="Enter your phone number">
```

```
</div>
```

```
<div class="form-group">
```

```
<label for="subject">Subject</label>
```

```
<select id="subject" required>
```

```
        <option value="">Select a subject</option>
        <option>General Inquiry</option>
        <option>Support</option>
        <option>Feedback</option>
    </select>
</div>

<div class="form-group">
    <label for="message">Message</label>

    <textarea id="message" rows="5" placeholder="Write your message..."
required></textarea>
</div>

<div class="form-group">
    <label for="file">Attachment</label>

    <input type="file" id="file">
</div>

    <button type="submit">Send Message</button>
</form>
</div>

</body>
</html>
```

9.A university registration page collects a student's name, email, password, confirm password, and age.

Using JavaScript, validate the form so that all fields are required, the email should not empty, the password is at least 8 characters long, the confirm password matches the password, and the age is a number greater than or equal to 18 before allowing submission.

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>University Registration</title>

<style>

  body {

    font-family: Arial, sans-serif;

    background: #f4f6f9;

    display: flex;

    justify-content: center;

    align-items: center;

    height: 100vh;

  }

  .form-box {

    background: #fff;

    padding: 25px;

    border-radius: 10px;
```

```
    box-shadow: 0 5px 15px rgba(0,0,0,0.1);  
    width: 300px;  
}
```

```
input {  
    width: 100%;  
    padding: 10px;  
    margin: 8px 0;  
    border: 1px solid #ccc;  
    border-radius: 5px;  
}
```

```
button {  
    width: 100%;  
    padding: 10px;  
    background: #007bff;  
    color: white;  
    border: none;  
    border-radius: 5px;  
    cursor: pointer;  
}
```

```
button:hover {  
    background: #0056b3;  
}
```

```
.error{
    color: red;
    font-size: 13px;
}
</style>
</head>
<body>

<div class="form-box">
    <h2>Register</h2>
    <form id="regForm" onsubmit="return validateForm()">
        <input type="text" id="name" placeholder="Full Name">
        <input type="email" id="email" placeholder="Email">
        <input type="password" id="password" placeholder="Password">
        <input type="password" id="confirmPassword" placeholder="Confirm Password">
        <input type="number" id="age" placeholder="Age">

        <p id="errorMsg" class="error"></p>

        <button type="submit">Register</button>
    </form>
</div>

<script>
function validateForm() {
    let name = document.getElementById("name").value.trim();
```

```
let email = document.getElementById("email").value.trim();

let password = document.getElementById("password").value;

let confirmPassword = document.getElementById("confirmPassword").value;

let age = document.getElementById("age").value;

let errorMsg = document.getElementById("errorMsg");


errorMsg.innerHTML = "";


// Check required fields
if (name === "" || email === "" || password === "" || confirmPassword === "" || age === "") {
    errorMsg.innerHTML = "All fields are required.";
    return false;
}


// Email validation (basic)
if (!email.includes("@")) {
    errorMsg.innerHTML = "Enter a valid email address.";
    return false;
}


// Password length
if (password.length < 8) {
    errorMsg.innerHTML = "Password must be at least 8 characters long.";
    return false;
}
```

```
// Confirm password match
if (password !== confirmPassword) {
    errorMsg.innerHTML = "Passwords do not match.";
    return false;
}

// Age validation
if (isNaN(age) || age < 18) {
    errorMsg.innerHTML = "Age must be 18 or above.";
    return false;
}

// If all validations pass
alert("Registration successful!");
return true;
}
</script>

</body>

</html>
```

10.Live Form Validation

Develop a registration form with email and password inputs. Use input event listeners on each field to validate in real-time: for email, check format show green/red icons; for password, enforce strength rules (length, characters) and update a error text. Clear errors on valid input and prevent form submission if invalid.

Register

<!-- Email -->

<div class="input-group">

<input type="text" id="email" placeholder="Email">

<div id="emailError" class="error"></div>

</div>

<!-- Password -->

<div class="input-group">

<input type="password" id="password" placeholder="Password">

<div id="passError" class="error"></div>

</div>

<button type="submit">Submit</button>

</form>